This is the first assignment for the Security course at LIACS. You can do this assignment **alone or in pairs**.

Submitting the assignment. In order to submit the assignment, you must first **join an "Assignment 1" group on BrightSpace by Thursday, March 5th 2026 at 23:59h at the latest.** You can find the groups from the course's home page by clicking on "Course tools → Groups". If you do the assignment in pairs, *both members of the group must join the same group.*

Once the deadline for joining groups has passed, you will be added to the assignment on ANS, where you will be able to enter you answers. You must **submit the assignment on ANS by Thursday, March 12th 2026 at 23:59h**. Note that submitting the assignment on ANS means *submitting on behalf of the whole group.* Multiple people can work together on the assignment at the same time (ANS saves progress whenever you navigate away from a question), but the submission is final for everyone.

1 Motivation

In this assignment you play the role of a nefarious hacker! You have just managed to exfiltrate (part of) the databases of five different companies. In these databases there are the usernames and password of the users of those companies, and you want to take over all of these accounts. However these companies have not stored the passwords in plaintext, but they have hashed them (you would be surprised by how many companies don't even do that in real life). Therefore your only way to find out what those passwords are, and get access to many important accounts, is to *bruteforce* them. Luckily you have been able to gather some additional data on each company, which should facilitate your task.

Disclaimer. Obviously, you should never do this for real! You should not access other people's data, independently of how bad their passwords are. Not even for "learning" or "finding vulnerabilities". In this assignment you work on some fake data, to test your knowledge of hashing and bruteforcing. Note that all the usernames, passwords, and other information that exist in this assignment have been generated randomly using a mix of Python's random module and the Faker library, therefore any similarities with real people are fortuitous.

2 Task

For this assignment, you are provided with a zip archive on BrightSpace. Inside, there are five folders corresponding to the five challenges that you will need to solve (the five "companies"). Inside each folder you will find a CSV file called `leveln.csv` (with n being 1 to 5): this file contains the "passwords" that you need to bruteforce. You can also find other files inside the folder, which can give you hints on how to solve the challenge.

Each challenge is slightly different from the others, so your strategy should be different for each. Also note that when your assignment is evaluated, it will be on different files that contain different content from what you are provided but follow the same logic (they will be generated with the same method but a different random seed). Therefore your goal is to find the logic needed to bruteforce each of the situations.

Your first task is to write a Python program that can solve all of these challenges. Your program should take two parameters: a number from 1 to 5 to indicate which of the challenges is being tried (so that you can write custom logic for each), and a path to the file that contains the "passwords" to hack. So for instance to ask your program to solve level 3 we will execute it as `./s123456789.py 3 /path/to/level3.csv` (assuming your program's name is `s123456789.py`).

Additionally, for each level we ask you to explain: (1) what is the logic that you used to bruteforce the passwords; (2) if you could only try one password every second, how long would it take *on average* for your program to hack a single password in that level?

3 Hashing in Python

In this assignment, all hashes are computed using the SHA256 hashing function. In order to compute such a hash in Python, you can use the following code:

```
from hashlib import sha256

text = "I want to hash this"
hasher = sha256()
hasher.update(text.encode("utf-8"))
print(hasher.hexdigest())  # Should print
↪  1ca71256c74a599cd302a3463c1a949560239a3f35a024e3657861fa81aa4efd
```

4 Submission requirements

Your submission must abide to the following requirements (use this list as a checklist):

- ☐ You have a **single** python script named with your student number (e.g., `s123456789.py`). You will need to upload this file in ANS. If you are working in pairs, name the file with the student number of any of the two members. We will make sure that both members receive the grade.
- ☐ Your script expects two arguments: a number from 1 to 5 to signal which level to solve, and the path to an input file for that level. You **must not** assume that any other particular file will be present on the machine that tests your program.
- ☐ Your script only uses libraries that are available in the Python **standard library**[1]. If you use any other library for development, take care of removing it before submission.
- ☐ For each username in the input file, your script prints a line with that username and the plaintext password, separated by a comma (`,`). Your script must print **nothing else**.
- ☐ Your script runs without errors. We suggest that you run the testing program (see next section) one final time before submission after doing any last-minute modifications.
- ☐ For each level you explain: (1) what is the logic that you used to solve it; (2) how long it would take your program on average to crack a single password, if it could only try one password per second. These answers will have to be entered in ANS.
- ☐ Your explanations are **in English and clear**.
- ☐ Your program is given a maximum time limit of 5 minutes to run. All the password sizes in this assignment are unrealistically small, enough that all levels can be solved altogether in less than a minute with the correct logic.

Part of our grading will be automated: if you do not follow the submission instructions your solution will not work correctly and therefore you will lose points.

5 Testing script

On BrightSpace you can also find a file named `test_submission.py`. If you execute this file from the directory of your program, it will automatically start your script and test all the different levels. It compares the output of your program with the expected output and tells you whether you managed to find the correct passwords for every user. *If this script's tests fail then your submission will fail*, so we suggest running it often on your program.

You might need to change the content of the `LEVELS_DIRECTORY` variable inside this script to point to where you extracted the archive.

[1] `https://docs.python.org/3/library/index.html`